

Game Design Document

Balloon Burst

Version 1.0 – December 2025

1. Game Overview

1.1 Title & Tagline

- **Title:** Balloon Burst
- **Tagline:** “Pop fast, dodge faster.”

1.2 High Concept

A fast-paced 2D arcade shooter where the player controls Goku on Nimbus, sliding left and right across the bottom of the screen and firing pins upward to pop a growing balloon before it becomes too large. Dragon Ball Z enemies act as moving blockers that disrupt your shots and force the player to reposition and time their attacks.

1.3 Short Description

Balloon Burst is a single-player 2D web game built in Unity. The game takes place on Supreme Kai's Planet and plays like a simple but demanding reaction-and-timing challenge.

The player moves Goku on Nimbus horizontally and shoots pins straight up. Each level features a balloon that drifts across the sky and slowly grows. If the balloon reaches a maximum size, the level restarts. If the player successfully pops it, they proceed to the next level and earn a score based on how small the balloon was at the moment of popping. DBZ enemies like Saibaman and Frieza move in the mid-air lane to block shots, creating timing and positioning puzzles.

A main menu and settings menu provide difficulty selection, volume adjustment, instructions, and access to a high-score table so players can try to beat their best runs.

2. Target Audience

2.1 Age Range & Demographics

- **Age:** 10+

- **Demographics:**

- Students in introductory game design courses (project context).
- Casual Dragon Ball Z fans who enjoy fan-made games.
- Players who like quick, arcade-style experiences that run in a web browser.

2.2 Player Types

- **Achievers**

- Chasing high scores and trying to pop the balloon at the smallest possible size.
- Beating the higher difficulty levels (Normal, Hard).

- **Skill-Driven Players (“Action Gamers”)**

- Enjoy mastering movement, timing, and dodging patterns.

- **Fans / Tourists**

- Attracted by the DBZ aesthetics and familiar characters.
- Want to experience the fantasy of Goku training on Supreme Kai’s Planet.

3. Genre & Platform

- **Genre**

- 2D Arcade Shooter / Action
- Light Score-Attack Game

- **Platform**

- Primary: WebGL build running in a desktop browser
- Engine: Unity 2022.3.x

- Controls: Keyboard only (Arrow Keys / WASD + Space / Ctrl)
-

4. Mechanics and Rules

4.1 Core Objective & Win Condition

- **Core Objective**
Pop the moving balloon before it grows too large, while avoiding enemy blockers.
- **Level Win Condition**
 - Pop the balloon with a pin → proceed to the next level.
- **Game Win Condition**
 - Complete all three levels on the chosen difficulty.
 - The game then returns to the main menu and records a high score for that run.

4.2 Game Components (Digital Objects)

Player (Goku on Nimbus)

- Moves horizontally at the bottom of the screen.
- Has a collider and Rigidbody2D for simple physics/interaction.
- Controlled via keyboard:
 - Left / Right Arrow or A / D – move.
 - Space or Left Ctrl – shoot pin.

Pin

- Instantiated from a pin prefab at the player's position.
- Moves straight upward at a constant speed.

- Gets destroyed when:
 - It hits the balloon (triggers a pop).
 - It hits an enemy (blocked).
 - It goes off-screen at the top.

Balloon

- Moves horizontally across the sky.
- Automatically grows over time (scale increases).
- Has a max size; exceeding it causes a level restart.
- On hit by a pin:
 - Calls `Pop()` → adds score and loads the next level.

Enemies (Distractors)

- Saibaman and Frieza sprites that move back and forth in mid-air.
- Their main role is to block pins, forcing players to re-position.
- Movement is continuous horizontal patrol between two X bounds.

UI & Menus

- **Main Menu:**
 - Game title ("Balloon Burst").
 - "Play Game" button.
 - "Settings" button.
 - "Instructions" button.
 - "High Scores" button.

- **Settings Menu:**

- Volume slider (controls global audio volume).
- Difficulty dropdown or buttons (Easy / Normal / Hard).
- Back to Main Menu button.

- **Instructions Panel:**

- Text explanation of controls, rules, and scoring.
- Back button.

- **In-Game HUD:**

- Score text: "Score: X".
- ESC → Pause / return to Main Menu (through `PauseMenuManager`).

High Score System

- Stores several top scores using `PlayerPrefs`.
- Displays high scores in a scrollable panel from the main menu.

4.3 Core Gameplay Loop

1. Choose difficulty and start the game from the main menu.
2. Level loads (Level 1, 2, or 3); player and balloon are placed, enemies spawn.
3. During play:
 - Player moves left/right to line up shots.
 - Player fires pins upward.
 - Balloon moves and gradually grows.
 - Enemies patrol and sometimes block shots.

4. Outcomes during the level:

- **If balloon is popped:**
 - Score is awarded based on balloon size.
 - Next level is loaded (or back to main menu if it was the final level).
 - **If balloon reaches max size:**
 - Level restarts (no score gain).
5. After Level 3 is beaten, total score is recorded, high-score list is updated, and the player returns to the main menu to replay.

4.4 Detailed Mechanics

4.4.1 Movement

- **Player**
 - Horizontal speed is tuned per difficulty.
 - Input uses Unity's new Input System (`Keyboard.current`).
- **Balloon Movement**
 - Uses `BalloonMovement` script:
 - Moves right until reaching `screenRight` bound, then flips X direction.
 - Moves left until reaching `screenLeft` bound, then flips back.
- **Enemies**
 - Use `DistractorMove` script:
 - `transform.Translate(Vector3.right * dir * speed * Time.deltaTime);`
 - Reverse direction when hitting `leftX` / `rightX` bounds.

4.4.2 Balloon Growth & Failure

- `BalloonGrowth` script:
 - `growRate` adjusted by difficulty:
 - Easy: slower growth.
 - Normal: base growth.
 - Hard: faster growth.
 - `InvokeRepeating(Grow, ...)` scales balloon by `Vector3.one * growRate`.
 - When `transform.localScale.x >= maxSize`:
 - Cancel growth.
 - Ask `GameManager` to restart the current level.

4.4.3 Scoring

- Implemented in `GameManager.AddScore(float balloonSize)`:
 - `points = Mathf.RoundToInt(100f / balloonSize);`
 - **Smaller balloon → more points.**
- Score persists across levels for the current run.
- High-score table saves final run score when all levels are finished.

4.4.4 Difficulty Settings

- Difficulty saved in a static `GameSettings` class.
- A difficulty UI element in Settings sets `GameSettings.difficultyIndex`:
 - `0` = Easy

- 1 = Normal
- 2 = Hard
- Difficulty affects:
 - Balloon growth rate (faster on hard).
 - Balloon speed and enemy count.

4.4.5 Levels

- **Level 1**
 - 1 balloon.
 - 1 distractor: Saibaman.
 - Intended as a tutorial / introduction.
- **Level 2**
 - 1 balloon.
 - 2 distractors: Saibaman + Frieza.
 - More screen coverage and blocking.
- **Level 3**
 - Balloon moves faster (and/or is smaller, making hits harder).
 - Same two distractors but tuned speeds → “hard mode”.
 - Final level; win here ends the run and records score.

4.5 Controls Summary

- **Move Left:** Left Arrow or A
- **Move Right:** Right Arrow or D

- **Shoot Pin:** Spacebar or Left Ctrl
 - **Pause / Main Menu:** ESC (via [PauseMenuManager](#))
-

5. Narrative and Setting

5.1 Story Premise

Goku is training on Supreme Kai's Planet to refine his precision and reaction speed. Rather than fighting directly, he's tasked with a special training drill: popping enchanted balloons before they grow unstable and explode. Meanwhile, enemies like Saibaman and Frieza appear as projections to interfere, forcing Goku to dodge and time his attacks perfectly.

5.2 Characters

- **Player Character – Goku on Nimbus**
 - Role: The only controllable character.
 - Personality: Determined, light-hearted in training mode.
- **Balloon**
 - Function: Training target that tests timing and aim.
 - Behavior: Calm at first, then increasingly threatening as it grows.
- **Saibaman**
 - Role: Early-game distractor/enemy.
 - Moves in predictable horizontal patterns.
- **Frieza**
 - Role: Late-game, higher-threat distractor.
 - Adds extra blocking and psychological pressure.

5.3 Progression & World-Building

Levels imply Goku's training intensifying:

- Level 1: Basic drill, one enemy.
- Level 2: Advanced drill, multiple enemies.
- Level 3: "Final test" – faster balloon, high-pressure environment.

The world is stylized as Supreme Kai's Planet: floating orbs, bright sky, soft hills and rocks in the background.

6. Art and Aesthetic Direction

6.1 Art Style

- 2D pixel/anime-inspired sprites with a DBZ look.
- Goku, enemies, and balloon are bright and readable silhouettes against a soft pastel background.
- Overall tone: energetic, playful training session rather than serious battle.

6.2 Tone & Mood

- Light-hearted, "training arc" atmosphere.
- Colorful background on Supreme Kai's Planet sets a mystical but optimistic mood.
- Audio:
 - Upbeat background music.
 - Popping sound effect for balloon hit.
 - Simple sound effects for shooting and menu navigation.

6.3 Color Palette & UI

- Background: sky blues, greens, and purples.

- Foreground: bright character sprites with high contrast.
- UI:
 - Orange “Score: X” text in top-right for visibility.
 - White buttons with solid colored outlines for Play, Settings, Instructions, etc.

6.4 User Interface Layout

- **Main Menu**
 - Title “Balloon Burst” centered over the background art.
 - Goku sprite in center as a focal point.
 - Buttons arranged vertically (Play Game, Settings, Instructions, High Scores).
- **In-Game HUD**
 - Score text in top-right.
 - Balloon and enemies occupy mid-air.
 - Player centered at bottom.
- **Settings & Instructions Panels**
 - Semi-transparent panels overlaying the background.
 - Simple text laid out in sections: Controls, Goals, Scoring, Difficulty.
 - Three levels, functioning menus, settings, animations, and high scores.
 - Polishing and deeper content could be extended in future work.

6.5 Tools Used

- Engine: **Unity 2022.3 LTS**
- Language: **C#**

- Libraries / Systems:
 - Unity's **new Input System** (`UnityEngine.InputSystem`).
 - **TextMeshPro** for UI text.
- Art: custom sprite assets and DBZ-inspired art assembled in Unity Sprite system.
- Version Control: Git + GitHub.

7.6 Key Scripts

- `PlayerMovement` – handles horizontal movement.
- `PlayerShoot` – instantiates and launches pins; also triggers player recoil animation.
- `PinMovement` – moves pins upward; handles out-of-bounds destruction.
- `BalloonMovement` – horizontal motion of the balloon between screen bounds.
- `BalloonGrowth` – balloon scaling, failure condition, scoring on pop.
- `DistractorMove` – enemy patrol logic.
- `PopOnContact` – collision logic when pins hit balloon/enemies.
- `GameManager` – scoring, level transition, game over logic, score animation trigger.
- `GameSettings` – global difficulty index, settings persistence.
- `UIManager` / `PauseMenuManager` – menu navigation, pause/resume, ESC handling.
- `MainMenu`, `SettingsMenu` – main and settings UI logic.
- `HighScoreManager` / `HighScoresUI` – record and display past scores.

8.7 Technical Details (Implementation)

- **Scenes in Build Settings (order):**

- MainMenu
 - Level 1
 - Level 2
 - Level 3
 - **Physics:**
 - 2D physics (Rigidbody2D + BoxCollider2D on player and balloon).
 - Simple collision checks for pin vs balloon/enemy via `OnTriggerEnter2D`.
 - **Input:**
 - New Input System via `Keyboard.current` in `Update()`.
 - **Audio:**
 - Volume slider writes to `AudioListener.volume` each frame or on change.
 - **Persistence:**
 - Difficulty and high scores stored in `PlayerPrefs`.
-

9. Testing and Feedback Summary

Key Issues Found & Solutions

Difficulty Setting Not Applying

- Issue: Changing difficulty to Normal or Hard always printed “Difficulty set to index: 0” and had no gameplay effect.
- Cause: Dropdown or UI element not correctly updating `GameSettings.difficultyIndex`.
- Solution:

- Hooked the difficulty dropdown's `OnValueChanged` event to a handler that sets `GameSettings.difficultyIndex` to the selected index.
- Added debug logs to confirm correct index per difficulty.
- Verified balloon grow rate changes by printing values on `Start()`.

ESC Key Not Returning to Main Menu on Level 2 and 3

- Issue: Pause/ESC worked in Level 1 but stopped working in later levels.
- Cause: `PauseMenuManager` or UI elements were only present / wired in Level 1 scene.
- Solution:
 - Duplicated and adjusted the pause menu canvas setup for Levels 2 and 3.
 - Ensured that the same script is present and listening for ESC in each level.
 - Tested ESC from all three levels.

Audio Slider Not Responding

- Issue: Volume slider in Settings visually changed but did not change actual game audio.
- Cause: Slider value wasn't connected to `AudioListener.volume`.
- Solution:
 - Connected slider's `OnValueChanged` event to a method that sets `AudioListener.volume`.
 - Confirmed that in-game audio reacts live to slider changes.

Balloon Growth Too Fast / Initial Size Too Large

- Issue: Balloon quickly became huge and filled the screen, giving players little time.
- Solution:

- Reduced `growRate` and `maxSize` in `BalloonGrowth`.
- Adjusted starting scale to be smaller.
- Increased the time between `Grow()` calls (e.g., every 4 seconds instead of 2.5).

Confusing Score Display Overlap

- Issue: Initial prototype showed an extra “0” overlapping with “Score: 0” in Level 2 or 3.
- Cause: Additional text object left over from an earlier version.
- Solution: Removed duplicate text in the scene and ensured only one score text is updated by `GameManager`.

Balloon Spawner Experiment (Abandoned)

- Issue: A `BalloonSpawner` object was tested to spawn multiple balloons for difficulty modes, but instances sometimes appeared only in Scene view, not Game view.
- Cause: Spawner references not correctly assigned; spawned balloons off-screen / behind camera, and conflicting with existing Balloon object.
- Solution: For this version, reverted to a single Balloon per level setup for reliability. Spawner concept left as future work.

10. Results of Playtesting, Player Feedback, and Revisions

10.1 Playtesting Observations

- New players quickly understood controls (move + shoot) when reading instructions.
- Most early failures were due to:
 - Ignoring enemies and having pins blocked.
 - Waiting too long and letting the balloon overgrow.
- Players naturally replayed to try:

- Beating all three levels on higher difficulty.
- Getting a better score by popping balloons earlier.

10.2 Player Feedback Highlights

- Positive:
 - “Feels like a mini DBZ training session.”
 - “Simple but satisfying when you finally time the shot around the enemy.”
 - “High score makes me want to retry.”
- Negative / Constructive:
 - Some players wanted clearer telegraphing that the balloon is close to failure (e.g., color or shake).
 - Hard difficulty felt abrupt when growth was too fast.
 - Volume slider originally didn’t seem to do anything.

10.3 Revisions Made

- Added clearer instructions panel in the main menu (controls, goal, scoring, explanation of difficulty).
 - Tuned balloon growth: slower on Easy, faster but still readable on Hard.
 - Fixed settings and ESC behavior across all scenes.
 - Began experimenting with visual feedback (balloon growth clarity and potential near-failure cues).
-

11. Animations

11.1 Goku Recoil Animation (Shoot Animation)

- **Animation Clip:** `GokuShoot`
- **Object:** Player (Goku on Nimbus)
- **Description:**
 - When Goku fires a pin, his sprite briefly **recoils**: a quick squash-and-stretch scale on X/Y combined with a small upward bob.
 - This gives visual punch to the shooting action.
- **Implementation:**
 - An Animator Controller `PlayerController` is attached to the Player.
 - `GokuShoot` is a state with keyframes that scale the player up slightly and then back to normal over ~0.2–0.3 seconds.

In `PlayerShoot.ShootPin()` (C#), after instantiating the pin, the script calls:

```
anim.Play("GokuShoot", 0, 0f);
```

- so the animation is explicitly started via code when the player actually shoots.

11.2 Score Text Animation (Score Bounce)

- **Animation Clip:** `ScoreBounce` (name can vary, but the concept is clear)
- **Object:** The TextMeshPro UI object that displays “Score: X”.
- **Description:**
 - Whenever the player gains points by popping the balloon, the score text does a quick bounce: it scales up slightly and then returns to normal size.
 - This provides feedback that the score changed and feels rewarding.
- **Implementation:**
 - A short animation clip scales the `scoreText` transform from `(1, 1, 1)` up to around `(1.2, 1.2, 1)` and back over ~0.25 seconds.

- Either an Animator or Animation component is attached to the score text object.

In `GameManager.AddScore(float balloonSize)`, after updating the text, code triggers the animation (for example):

```
// after scoreText.text = ...
var anim = scoreText.GetComponent<Animator>();
if (anim != null)
{
    anim.Play("ScoreBounce", 0, 0f);
}
```

12. Extra Technical Details

To make the implementation concrete:

- **Resolution / Aspect:**
 - Designed for wide screens (e.g., 16:9). Canvas uses **Scale with Screen Size** so UI stays readable.
 - **Layers & Sorting:**
 - Background, Midground, Player, and UI each use separate sorting layers.
 - “Player” and “Balloon” sprites are on higher layers to stay visually on top.
 - **Prefabs:**
 - Balloon, Pin, Saibaman, Frieza, and some UI elements are prefabs so they can be reused across levels.
 - **Build Target:**
 - WebGL with compressed data (Unity default WebGL template).
 - A zipped WebGL build is included in the repo for submission.
-

13. Resources and Citations (Programming / Unity Help)

Below is what I used for **programming / Unity help**.

13.1 ChatGPT (Programming & Design Help)

I used **ChatGPT (OpenAI)** multiple times while building Balloon Burst.

Example – Fixing bugs and difficulty logic

- **Prompt (shortened):**

“Another problem, when I change the difficulty level to normal or hard, it doesn’t change anything. The console always says "Difficulty set to index: 0.”

- **Output (summary):**

- Explained that the dropdown’s `OnValueChanged` must be wired correctly.
- Suggested logging the selected index and showed how to store it in a static `GameSettings` class.

I used similar prompts for:

- Making the ESC key go back to main menu.
- Fixing the audio slider so it actually changed `AudioListener.volume`.
- Helping with Git / GitHub commands to push my Unity project.

13.2 Other Resources

- Unity Documentation – **SceneManager**
 - <https://docs.unity3d.com/ScriptReference/SceneManagement.SceneManager.html>
 - Used to look up how to load scenes by build index and reload the current scene.
- Unity Documentation – **Input System / Keyboard**

- <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/index.html>
- Used to confirm usage of `Keyboard.current` and `wasPressedThisFrame`.
- Unity Documentation – **TextMeshPro**
 - <https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/manual/index.html>
 - Used for setting up score text and instructions text.